Indian Institute of Technology Kanpur

Department of Computer Science and Engineering

CS614: Linux Kernel Programming

Final Report

Analysis, Benchmarking, and Optimization of ext4/JBD2 Consistency Mechanism on Linux 6.1.4

Team Members

Name	**Roll Number**
Milan Roy	241110042
Sahil Basia	241110061
Shrey Sharma	251110068
Suyamoon Pathak	241110091

April 2026

Abstract

This report describes the work of the **MTech Rocks** group on the ext4/JBD2 consistency mechanism on Linux 6.1.4. We benchmarked the baseline cost of each ext4 journaling mode and identified where the journal hurts performance most; this characterization provided clear directions for further optimization. We then addressed two fast-commit fallback paths with small kernel patches: `EXT4_FC_REASON_XATTR` (108 lines, 64× fewer JBD2 full commits on a setxattr workload, 27% wall-time gain on VM and 48% on bare-metal) and `EXT4_FC_REASON_FALLOC_RANGE` (46 lines, 62.5× fewer commits on collapse/insert range, 23–26% improvement on VM and 42–44% on bare-metal).

We further developed a kernel module using `kretprobes` to measure that ∼78% of fsync time is spent inside `jbd2_log_wait_commit`, and replaced this function with a lockless CAS-based version, achieving improvements of +40%, +54%, and +7% IOPS at 16 threads across three workloads, without regression in streaming performance.

Finally, we designed a kernel module that replaces JBD2's strict ring-buffer journal allocator with a free-block bitmap, reducing P99 latency on a WAL workload from 0.41 ms to 0.05 ms in `ordered` mode while maintaining the same write amplification factor.

Contents

1 Introduction

Modern filesystems must guarantee consistency in the face of unexpected system crashes or power failures. Without such guarantees, an interrupted write sequence can leave the filesystem in a partially updated state, corrupting metadata structures such as inodes, directory entries, and block allocation bitmaps. Recovery from such corruption is expensive, often requiring a full filesystem check (`fsck`), and may result in permanent data loss.

To address this, ext4 — the default filesystem on most Linux distributions — employs a journaling layer called **JBD2** (Journaling Block Device 2). Rather than writing updates directly to their final on-disk locations, JBD2 first records changes atomically to a dedicated circular log called the *journal*. On crash recovery, the kernel replays any committed but not yet checkpointed transactions from the journal, restoring the filesystem to a consistent state without a full scan.

JBD2 organises writes into *transactions*: a group of modifications that are committed atomically. Each transaction passes through a well-defined pipeline — accumulating updates, locking for commit, writing descriptor and data blocks to the journal, issuing a commit block, and finally checkpointing dirty blocks to their home locations. Crucially, a transaction is not considered durable until a commit block is flushed to stable storage, enforcing the write-ahead property.

Ext4 exposes three journaling modes, each offering a different trade-off between consistency and performance:

- **Journal mode**: Both file data and metadata are written to the journal before being committed to their final locations. This provides the strongest consistency guarantees — even file data is recoverable after a crash — but incurs the highest overhead due to the double-write of all data blocks.
- **Ordered mode** (default): Only metadata is journaled. However, ext4 enforces a strict ordering constraint: data blocks must be flushed to their final locations *before* the corresponding metadata transaction is committed to the journal. This prevents stale data from being exposed after a crash, while avoiding the cost of journalling data blocks directly.
- **Writeback mode**: Only metadata is journaled, with no ordering constraint between data and metadata writes. This mode offers the best write throughput but the weakest consistency guarantee: after a crash, a committed metadata update may point to a data block that was never fully written, potentially exposing stale or garbage data.

2 Workload Characterization

3 Workload Characterisation

3.1 Sync-Heavy Small Writes Workload

3.1.1 Workload Description

This workload emulates the write-ahead logging (WAL) pattern typical of database systems, in which each log record must be made durable before the next write can proceed. Using `fio`, a single thread writes a total of 100 MB to a file in fixed-size blocks of 4, 8, 16, 32, or 64 KB. An `fsync` is issued after every block write, forcing a JBD2 transaction commit to complete before the subsequent write begins. The benchmark is executed across all three ext4 journaling modes — `ordered`, `journal`, and `writeback` — together with a `nojournal` baseline, enabling a controlled comparison of journaling overhead and write amplification across block sizes. Each configuration is repeated five times and the results are averaged to mitigate measurement variability.

Table 1: Throughput, Latency, and I/O Breakdown across Journaling Modes and Block Sizes

Mode	BS	IOPS	BW (MB/s)	Runtime (s)	Avg lat. (ms)	write() (µs)	fsync() (µs)	fsync %
ordered	4K	185 ± 4	0.72 ± 0.01	138.05 ± 2.70	5.39 ± 0.11	20 ± 0	5369 ± 105	99.6%
	8K	180 ± 5	1.40 ± 0.04	71.24 ± 2.20	5.56 ± 0.17	22 ± 0	5541 ± 172	99.6%
	16K	174 ± 4	2.71 ± 0.06	36.85 ± 0.83	5.76 ± 0.13	25 ± 0	5731 ± 130	99.6%
	32K	173 ± 2	5.39 ± 0.06	18.55 ± 0.19	5.80 ± 0.06	28 ± 0	5767 ± 61	99.5%
	64K	170 ± 3	10.62 ± 0.21	9.41 ± 0.19	5.89 ± 0.12	39 ± 0	5845 ± 119	99.3%
journal	4K	179 ± 3	0.70 ± 0.01	142.78 ± 2.63	5.57 ± 0.10	23 ± 0	5551 ± 103	99.6%
	8K	169 ± 2	1.32 ± 0.01	75.63 ± 0.71	5.91 ± 0.06	26 ± 0	5880 ± 55	99.6%
	16K	166 ± 3	2.59 ± 0.04	38.65 ± 0.60	6.04 ± 0.09	31 ± 0	6006 ± 93	99.5%
	32K	165 ± 3	5.17 ± 0.11	19.36 ± 0.41	6.05 ± 0.13	40 ± 0	6007 ± 129	99.3%
	64K	154 ± 7	9.62 ± 0.47	10.41 ± 0.51	6.51 ± 0.32	59 ± 1	6447 ± 320	99.1%
writeback	4K	185 ± 2	0.72 ± 0.01	138.21 ± 1.33	5.40 ± 0.05	20 ± 0	5376 ± 52	99.6%
	8K	181 ± 1	1.41 ± 0.01	70.88 ± 0.27	5.53 ± 0.02	22 ± 0	5513 ± 21	99.6%
	16K	170 ± 1	2.66 ± 0.02	37.58 ± 0.28	5.87 ± 0.04	25 ± 0	5844 ± 44	99.6%
	32K	160 ± 7	5.00 ± 0.22	20.05 ± 0.88	6.26 ± 0.27	29 ± 0	6234 ± 275	99.5%
	64K	159 ± 6	9.93 ± 0.40	10.08 ± 0.43	6.30 ± 0.27	40 ± 1	6262 ± 268	99.4%
nojournal	4K	381 ± 3	1.49 ± 0.01	67.17 ± 0.56	2.62 ± 0.02	15 ± 0	2604 ± 22	99.4%
	8K	362 ± 5	2.83 ± 0.04	35.38 ± 0.50	2.76 ± 0.04	17 ± 0	2743 ± 39	99.4%
	16K	296 ± 5	4.62 ± 0.08	21.63 ± 0.37	3.38 ± 0.06	20 ± 0	3356 ± 59	99.4%
	32K	307 ± 8	9.60 ± 0.26	10.43 ± 0.28	3.26 ± 0.09	24 ± 0	3230 ± 86	99.3%
	64K	269 ± 13	16.82 ± 0.79	5.95 ± 0.29	3.72 ± 0.18	34 ± 0	3684 ± 181	99.1%

3.1.2 Throughput and Latency

Prior to benchmarking, we expected `ordered` mode to exhibit slightly lower IOPS than `writeback` mode, owing to the additional ordering constraint that data blocks must be flushed to disk before their corresponding metadata is committed to the journal. We further expected `journal` mode to incur substantially lower IOPS than either of the

other two modes, given the overhead of writing file data into the journal in addition to metadata. Table 1 shows neither expectation to be borne out: IOPS across all three journaling modes are remarkably similar, whilst `nojournal` achieves roughly twice the IOPS of any journaled mode.

The near-equivalence among the three journaled modes follows from the fact that all three must traverse the same JBD2 commit pipeline on every `fsync`: an NVMe cache flush to persist both data and journal log blocks to their on-disk locations, followed by a direct write of the commit block to stable storage. The additional data written into the journal by `journal` mode contributes only a marginal cost at smaller block sizes, insufficient to produce a measurable IOPS gap. `nojournal` mode, by contrast, incurs only the first of these two costs — a single NVMe cache flush — and writes no journal entries whatsoever, halving the per-`fsync` barrier count and eliminating journal write amplification entirely. This accounts for why `nojournal` achieves approximately twice the IOPS of the journaled modes, particularly at smaller block sizes. As block size increases, IOPS decline modestly across all modes, reflecting the growing per-operation data transfer cost beginning to contribute alongside the fixed flush overhead.

Although per-I/O latency and IOPS remain roughly constant across block sizes for a given mode, total benchmark time roughly halves with each doubling of block size. Doubling the block size doubles the data written per `fsync` call, so the total number of `fsync` calls required is halved proportionally. Since the per-`fsync` cost is dominated by the fixed NVMe flush latency, total time falls in proportion. Notably, total time is consistent across all journaling modes at a given block size, including `journal` mode, for the same reason: the additional data journaled by `journal` mode does not meaningfully increase the per-flush cost.

3.1.3 Impact of fsync

As shown in Table 1, the `write()` system call contributes at most 0.9% of total per-I/O latency across all modes and block sizes, since it merely copies data into the page cache and returns immediately. The `write()` latency increases slightly with block size — most visibly in `journal` mode — but does not double when the block size doubles, indicating a fixed component.

The `fsync()` latency, by contrast, is two orders of magnitude higher and subsumes the cost of the JBD2 commit pipeline and the NVMe cache flush. `fsync()` similarly does not double with block size, reflecting the large fixed component of NVMe flush latency that is independent of the volume of data being flushed, at least at smaller block sizes. Consequently, `fsync()` dominates total per-I/O latency in every configuration, conclusively establishing that the bottleneck is the mandatory durability guarantee enforced by `fsync()`, not the movement of data into the page cache.

3.1.4 Physical Bytes and Write Amplification

Table 2: Write Amplification, Per-Transaction Workload, and JBD2 Commit Pipeline across Journaling Modes and Block Sizes

Mode	BS	Physical (MB)	Extra Bytes (MB)	WAF	Total txns	Blocks logged/txn	Avg Logging (ms)	Avg E2E commit (ms)	JBD2 commit %
ordered	4K	400.92 ± 1.79	300.92 ± 1.79	2.0065 ± 0.0090	25600 ± 1	3.0 ± 0.0	5.268 ± 0.102	5.272 ± 0.102	$97.8 \pm 1.9\%$
	8K	250.08 ± 0.01	150.08 ± 0.01	1.6670 ± 0.0001	12801 ± 1	3.0 ± 0.0	5.437 ± 0.172	5.441 ± 0.172	$97.8 \pm 3.1\%$
	16K	175.06 ± 0.01	75.06 ± 0.01	1.4002 ± 0.0000	6401 ± 0	3.0 ± 0.0	5.621 ± 0.129	5.625 ± 0.129	$97.7 \pm 2.2\%$
	32K	137.56 ± 0.00	37.56 ± 0.00	1.2225 ± 0.0000	3201 ± 0	3.0 ± 0.0	5.651 ± 0.065	5.655 ± 0.065	$97.6 \pm 1.1\%$
	64K	118.81 ± 0.00	18.81 ± 0.00	1.1180 ± 0.0000	1601 ± 0	3.0 ± 0.0	5.718 ± 0.119	5.722 ± 0.119	$97.3 \pm 2.0\%$
journal	4K	500.14 ± 0.00	400.14 ± 0.00	2.5031 ± 0.0008	25600 ± 0	4.0 ± 0.0	5.460 ± 0.100	5.467 ± 0.100	$98.0 \pm 1.8\%$
	8K	350.10 ± 0.00	250.10 ± 0.00	2.3336 ± 0.0001	12800 ± 0	5.0 ± 0.0	5.778 ± 0.051	5.785 ± 0.051	$97.9 \pm 0.9\%$
	16K	275.07 ± 0.00	175.07 ± 0.00	2.2001 ± 0.0000	6400 ± 0	7.0 ± 0.0	5.896 ± 0.093	5.903 ± 0.093	$97.7 \pm 1.5\%$
	32K	237.54 ± 0.00	137.54 ± 0.00	2.1111 ± 0.0000	3200 ± 0	11.0 ± 0.0	5.889 ± 0.130	5.897 ± 0.130	$97.5 \pm 2.1\%$
	64K	218.76 ± 0.00	118.76 ± 0.00	2.0584 ± 0.0000	1600 ± 0	19.0 ± 0.0	6.315 ± 0.318	6.323 ± 0.318	$97.2 \pm 4.9\%$
writeback	4K	400.11 ± 0.01	300.11 ± 0.01	2.0024 ± 0.0006	25600 ± 0	3.0 ± 0.0	5.275 ± 0.051	5.279 ± 0.051	$97.8 \pm 1.0\%$
	8K	250.09 ± 0.00	150.09 ± 0.00	1.6670 ± 0.0001	12801 ± 0	3.0 ± 0.0	5.406 ± 0.023	5.410 ± 0.023	$97.7 \pm 0.4\%$
	16K	175.06 ± 0.01	75.06 ± 0.01	1.4002 ± 0.0001	6401 ± 1	3.0 ± 0.0	5.731 ± 0.044	5.735 ± 0.044	$97.7 \pm 0.8\%$
	32K	137.56 ± 0.00	37.56 ± 0.00	1.2225 ± 0.0000	3201 ± 0	3.0 ± 0.0	6.112 ± 0.272	6.116 ± 0.272	$97.7 \pm 4.3\%$
	64K	118.81 ± 0.00	18.81 ± 0.00	1.1180 ± 0.0000	1601 ± 0	3.0 ± 0.0	6.130 ± 0.265	6.135 ± 0.265	$97.4 \pm 4.2\%$
nojournal	4K	199.81 ± 0.06	99.81 ± 0.06	1.0000 ± 0.0004	—	—	—	—	—
	8K	150.02 ± 0.01	50.02 ± 0.01	1.0000 ± 0.0001	—	—	—	—	—
	16K	125.02 ± 0.00	25.02 ± 0.00	1.0000 ± 0.0000	—	—	—	—	—
	32K	112.52 ± 0.00	12.52 ± 0.00	1.0000 ± 0.0000	—	—	—	—	—
	64K	106.27 ± 0.00	6.27 ± 0.00	1.0000 ± 0.0000	—	—	—	—	—

The Physical and Extra Bytes columns in Table 2 capture the disk-write overhead beyond the 100 MB logical payload, as measured by `blktrace`. An important detail when interpreting these values is that disk writes are granular to 4 KB: even a few bytes of metadata require a full 4 KB block, inflating the observed overhead accordingly.

Across all modes, extra bytes halve as block size doubles, since the transaction count halves proportionally and each transaction carries a fixed metadata footprint. In `journal` mode this halving is partially masked by an additional $\approx$100 MB term arising from the data blocks being written twice into the journal, but removing this constant reveals the same underlying pattern.

For `ordered` and `writeback` modes, which log exactly three blocks per transaction — one descriptor, one metadata, and one commit block — the extra bytes can be verified analytically:

$$3 \times 25{,}600 \text{ transactions} \times 4\,\text{KB} \approx 300\,\text{MB} \tag{1}$$

This matches the observed value at 4 KB, and the same halving applies at larger block sizes. An analogous calculation for `journal` mode must additionally account for the data blocks logged per transaction, which scale linearly with block size.

WAF (Table 2) quantifies how many times more data is written to disk relative to the `nojournal` baseline. For `ordered` and `writeback`, WAF decreases from 2.00$\times$ at 4 KB to 1.12$\times$ at 64 KB, as the fixed three-block journal overhead is progressively diluted by the growing data payload. `journal` mode converges far more slowly, from 2.50$\times$ to 2.06$\times$, because its data-journaling cost scales with block size and cannot be diluted in the same manner.

3.1.5 JBD2 Commit Percentage of Benchmark Time

As shown in Table 2, the average logging time — the time spent writing the journal transaction to the NVMe device — accounts for virtually all of the end-to-end commit time across every configuration. JBD2 commit activity consequently dominates total benchmark wall-clock time, a direct consequence of the `fsync`-after-every-write pattern that chains each write to the completion of the preceding commit. With the application blocked almost entirely on synchronous I/O, there is no opportunity for any form of parallelism or batching.

3.1.6 Conclusion

The sync-heavy WAL workload establishes that, under a per-write `fsync` discipline, the dominant cost is the NVMe cache flush that `fsync()` must issue on every commit, not the movement of data or metadata within the journaling layer. All three journaled modes exhibit near-identical IOPS because they share the same commit pipeline; the additional data written by `journal` mode is insufficient to widen this gap at the block sizes examined. `nojournal` mode achieves approximately twice the IOPS by eliminating the journal write entirely and incurring only a single flush per `fsync`. WAF decreases with block size for `ordered` and `writeback` as the fixed three-block overhead is diluted by larger payloads, whereas `journal` mode's WAF converges only slowly because its data-journaling cost grows with block size.

3.2 Sequential Write Workload

3.2.1 Workload Description

This workload measures the raw sequential write performance of each ext4 journaling mode without the durability constraint of a per-write `fsync`, emulating the log-writing behaviour common in database and application logging scenarios. `fio` is configured as a single-threaded workload writing a total of 10 GB of data to a file in fixed-size blocks of 128 KB, 256 KB, 512 KB, and 1 MB. Unlike the sync-heavy WAL workload, no `fsync` is issued between writes, allowing the kernel to freely batch dirty pages and pipeline writes to disk. JBD2 commits are therefore not driven by the application but are instead triggered by the kernel's background commit timer, which fires every 5 seconds to lock the current running transaction and flush it to disk. The benchmark is executed across all three ext4 journaling modes — `ordered`, `journal`, `writeback`, and `nojournal` — to isolate the cost of each mode's journaling policy under conditions where the kernel is free to amortise overhead across large batches of writes. Each configuration is repeated five times and the results are averaged to mitigate measurement variability.

3.2.2 Throughput and Latency

Table 3: Throughput, Latency, and I/O Breakdown across Journaling Modes and Block Sizes (Sequential Write)

Mode	BS	IOPS	BW (MB/s)	Runtime (s)	Avg (ms)	write() (µs)
ordered	128K	1580 ± 772	197.54 ± 96.47	65.35 ± 42.10	0.80 ± 0.51	796 ± 514
	256K	693 ± 160	173.17 ± 40.01	61.14 ± 13.11	1.49 ± 0.32	1488 ± 319
	512K	491 ± 81	245.67 ± 40.48	42.49 ± 7.41	2.07 ± 0.36	2066 ± 361
	1M	101 ± 22	100.94 ± 22.16	105.32 ± 26.46	10.28 ± 2.58	10251 ± 2584
journal	128K	290 ± 7	36.22 ± 0.93	282.83 ± 7.26	3.45 ± 0.09	3451 ± 89
	256K	147 ± 4	36.86 ± 0.92	277.89 ± 7.04	6.78 ± 0.17	6781 ± 172
	512K	74 ± 4	37.17 ± 2.12	276.08 ± 16.28	13.48 ± 0.80	13473 ± 796
	1M	28 ± 0	27.96 ± 0.20	366.30 ± 2.69	35.77 ± 0.26	35750 ± 262
writeback	128K	993 ± 44	124.15 ± 5.55	82.59 ± 3.61	1.01 ± 0.04	1006 ± 44
	256K	520 ± 8	130.03 ± 1.92	78.76 ± 1.15	1.92 ± 0.03	1919 ± 28
	512K	270 ± 21	134.80 ± 10.46	76.26 ± 5.78	3.72 ± 0.28	3712 ± 284
	1M	82 ± 1	81.73 ± 0.93	125.30 ± 1.42	12.23 ± 0.14	12201 ± 139
nojournal	128K	1362 ± 529	170.24 ± 66.07	67.92 ± 30.58	0.83 ± 0.37	826 ± 373
	256K	714 ± 67	178.47 ± 16.69	57.72 ± 5.60	1.41 ± 0.14	1405 ± 137
	512K	413 ± 106	206.60 ± 52.97	51.59 ± 11.93	2.52 ± 0.58	2508 ± 582
	1M	96 ± 2	95.77 ± 2.10	106.95 ± 2.33	10.44 ± 0.23	10407 ± 232

The most striking feature of Table 3 is the sharp and persistent throughput gap between `journal` mode and the remaining three modes across all block sizes. Without a per-write `fsync` serialising every commit, the kernel is free to batch dirty pages and commit journal transactions in the background without blocking the application thread: page cache writeback and JBD2 transaction commits proceed concurrently with the benchmarked workload, and since no explicit flush command is issued to the NVMe device, the cache flush cost is absorbed in parallel as well. This removes the dominant bottleneck of the sync-heavy workload and exposes the true cost of each mode's journaling policy in isolation.

A notable contrast with the sync-heavy workload is that total benchmark time in Table 3 is largely independent of block size for `ordered`, `writeback`, and `nojournal`. In the sync-heavy workload, total time halved with each doubling of block size because each `fsync` incurred a fixed NVMe flush cost and larger blocks reduced the number of `fsync` calls required. Here, with no per-write `fsync`, the flush cost is no longer on the critical path: the workload, page cache writeback, and journal commits all proceed concurrently, keeping the NVMe device continuously busy regardless of block size. This parallelism also explains why the IOPS, bandwidth, and `write()` latency of `ordered` and `writeback` remain close to the `nojournal` baseline — the journaling overhead of these modes is absorbed entirely in the background and does not delay the application thread.

3.2.3 Write Amplification and Commit Behaviour

Table 4: Write Amplification and JBD2 Commit Behaviour across Journaling Modes and Block Sizes (Sequential Write)

Mode	BS	Physical (MB)	WAF	Total Commits	Total Time (s)	Commit interval (s)
ordered	128K	10240.94 ± 0.13	1.0000 ± 0.0000	16 ± 8	65.35 ± 42.10	4.084 ± 3.331
	256K	10240.92 ± 0.03	1.0000 ± 0.0000	14 ± 2	61.14 ± 13.11	4.367 ± 1.125
	512K	10240.87 ± 0.02	1.0000 ± 0.0000	11 ± 1	42.49 ± 7.41	3.863 ± 0.760
	1M	10241.04 ± 0.08	1.0000 ± 0.0000	22 ± 5	105.32 ± 26.46	4.787 ± 1.622
journal	128K	20522.52 ± 1.22	2.0040 ± 0.0001	622 ± 42	282.83 ± 7.26	0.455 ± 0.033
	256K	20523.58 ± 0.94	2.0041 ± 0.0001	646 ± 0	277.89 ± 7.04	0.430 ± 0.011
	512K	20522.95 ± 0.63	2.0041 ± 0.0001	646 ± 0	276.08 ± 16.28	0.427 ± 0.025
	1M	20523.61 ± 0.26	2.0041 ± 0.0001	646 ± 0	366.30 ± 2.69	0.567 ± 0.004
writeback	128K	10240.99 ± 0.02	1.0000 ± 0.0000	18 ± 2	82.59 ± 3.61	4.588 ± 0.548
	256K	10240.98 ± 0.01	1.0000 ± 0.0000	18 ± 1	78.76 ± 1.15	4.376 ± 0.251
	512K	10240.97 ± 0.01	1.0000 ± 0.0000	18 ± 1	76.26 ± 5.78	4.237 ± 0.398
	1M	10241.10 ± 0.02	1.0000 ± 0.0000	26 ± 1	125.30 ± 1.42	4.819 ± 0.193
nojournal	128K	10240.56 ± 0.08	1.0000 ± 0.0000	—	67.92 ± 30.58	—
	256K	10240.58 ± 0.01	1.0000 ± 0.0000	—	57.72 ± 5.60	—
	512K	10240.59 ± 0.05	1.0000 ± 0.0000	—	51.59 ± 11.93	—
	1M	10240.70 ± 0.24	1.0000 ± 0.0000	—	106.95 ± 2.33	—

Table 4 consolidates the physical write volume, WAF, and commit behaviour for each mode. Unlike the sync-heavy workload, where physical bytes written decreased as block size doubled due to reduced per-transaction overhead, here the physical byte count is essentially constant across all block sizes within each mode. `ordered`, `writeback`, and `nojournal` each produce approximately 10.240 GB of physical writes, whilst `journal` mode consistently produces around 20.522 GB. This block-size independence arises because JBD2 commits are driven by the background timer rather than by per-write `fsync` calls, leaving the physical write volume determined entirely by the data volume and the mode's journaling policy rather than by write granularity.

The WAF results reflect this directly. `ordered` and `writeback` achieve a WAF of effectively $1.0000\times$ across all block sizes, identical to `nojournal`. With the background timer batching thousands of writes per transaction — only 14–28 total commits are issued across the entire 10 GB run — the fixed per-transaction overhead of descriptor, metadata, and commit blocks is amortised to negligible levels. This contrasts sharply with the sync-heavy workload, where `ordered` mode carried a WAF of $2.00\times$ at 4 KB because every write forced its own individual commit. `journal` mode, by contrast, maintains a WAF of exactly $2.004\times$ across every block size with no variation. Because every data block must be written into the journal in addition to its final disk location, the extra write cost is strictly proportional to the volume of data written rather than to the number of transactions, and no amount of JBD2 batching can reduce this inherent $2\times$ factor.

The commit interval column makes the underlying commit mechanism explicit.

`ordered` and `writeback` show commit intervals of 4.0–4.8 s across all block sizes, consistent with the kernel's 5 second background commit timer firing with no other trigger. `journal` mode, by contrast, exhibits a commit interval of only 0.427–0.567 s, roughly 20× more frequent than the timer period. This is because each transaction in `journal` mode must accommodate both data and metadata blocks, and the journal's fixed maximum transaction capacity is exhausted long before the 5 second timer fires. Once the transaction fills its block allocation, JBD2 forces an immediate commit regardless of elapsed time, resulting in 622–646 commits over the course of the benchmark compared to only 11–26 for the other journaled modes.

3.2.4 Conclusion

The sequential write workload exposes a fundamental asymmetry in how the four journaling modes respond to the removal of the per-write durability constraint. `ordered` and `writeback` exploit JBD2 transaction batching to amortise their per-transaction journal overhead to negligible levels, achieving near-identical bandwidth to `nojournal` and a WAF of effectively 1.0000× across all block sizes. `journal` mode is structurally unable to benefit from this batching: its 2× data amplification is proportional to the volume of data written rather than to the number of transactions, resulting in a constant WAF of 2.004×, a commit interval 20× shorter than the background timer period, and a persistent throughput penalty regardless of block size or transaction depth. For workloads that do not require per-write durability, `journal` mode should therefore be avoided, as its throughput penalty is inherent to its data-journaling policy and cannot be mitigated by tuning block size or commit interval.

3.3 Combined Workload

The two preceding workloads represented opposite extremes of kernel commit behaviour: one in which a JBD2 commit is forced after every write, and one in which commit timing is delegated entirely to the kernel's background timer. This workload places both patterns under simultaneous pressure, running a sync-heavy and a sequential-write benchmark concurrently so that their interactions can be observed under realistic mixed-pressure conditions. The sync-heavy component issues 8 KB writes each followed by an `fsync`, whilst the sequential-write component operates with a 1 MB block size. Unlike the earlier workloads, which were bounded by a fixed data volume, both components here run for a fixed duration of two minutes. The benchmark is executed across all four ext4 journaling modes to isolate the overhead attributable to each mode's journaling policy. Every configuration is repeated five times and the results are averaged to mitigate measurement variability.

Table 5: Combined Throughput and Average Latency for WAL and Sequential Workloads

Workload	Mode	IOPS	BW (MB/s)	Data (GB)	Avg lat. (ms)
SYNC	ordered	4 ± 1	0.03 ± 0.01	0.00 ± 0.00	252.97 ± 79.97
	journal	4 ± 0	0.03 ± 0.00	0.00 ± 0.00	228.39 ± 18.54
	writeback	2 ± 1	0.02 ± 0.01	0.00 ± 0.00	493.88 ± 254.09
	nojournal	8 ± 5	0.06 ± 0.04	0.01 ± 0.00	165.53 ± 98.02
SEQ	ordered	107 ± 11	106.74 ± 11.07	12.52 ± 1.30	8.83 ± 0.59
	journal	34 ± 2	34.29 ± 2.12	4.03 ± 0.25	29.25 ± 1.69
	writeback	109 ± 17	108.61 ± 16.58	12.83 ± 1.93	9.08 ± 1.68
	nojournal	96 ± 13	95.70 ± 13.44	11.33 ± 1.43	10.07 ± 1.60

3.3.1 Throughput and Latency

Comparing Table 5 with Table 1, the WAL component suffers a severe degradation under concurrent load. IOPS collapse from the ∼170–381 range observed in isolation to just 2–8, a reduction of 95–99% across all modes. Bandwidth falls correspondingly from up to ∼16.8 MB/s to below 0.06 MB/s, and average latency rises dramatically: in `ordered` mode, for instance, it increases from ∼5.4 ms to ∼253 ms, a 46× inflation. Comparable degradation is observed across all other journaling modes.

The mechanism is a system-wide coupling introduced by `fsync`. When the WAL component calls `fsync`, the kernel does not flush only the pages belonging to the WAL file: it locks the current JBD2 transaction and forces a commit of all outstanding dirty data from every writer sharing the filesystem, including the large dirty page footprint accumulated by the concurrently running sequential workload. The `fsync` call therefore cannot return until this much larger volume of data has been written back and journaled, inflating each individual `fsync` latency by roughly two orders of magnitude relative to its isolated value. Because the WAL thread is blocked for the entirety of each `fsync`, its effective throughput collapses.

The sequential workload, by contrast, shows only moderate changes relative to Table 3. IOPS and bandwidth are marginally higher in some modes — approximately 5–15% for `ordered` and `writeback` — whilst average latency decreases slightly. This modest improvement arises because the WAL component's periodic `fsync` calls trigger forced JBD2 commits that writeback the sequential workload's dirty pages as a side effect, reducing the residual writeback work that would otherwise fall to the background flusher and slightly lowering the `write()` latency seen by the sequential thread.

3.3.2 Conclusion

The combined workload reveals a cross-workload interference effect that is absent in either workload individually: a single `fsync`-issuing thread can severely degrade its own throughput by inadvertently taking on the writeback cost of all co-resident writers. The WAL component's `fsync` latency inflates by up to 46× relative to its isolated value because each commit is forced to drain the full dirty page set of the concurrently running

sequential workload. This interference is an intrinsic property of the JBD2 transaction model, in which a commit is a filesystem-wide operation rather than a per-file one, and its severity scales with the volume of dirty data accumulated by other writers at the time `fsync` is called. The sequential workload is largely unaffected and even benefits slightly, as the WAL-driven commits serve as additional writeback opportunities that marginally reduce its own background flusher load. These results highlight that, in multi-workload environments, the performance of any `fsync`-dependent application is sensitive not only to its own I/O pattern but to the aggregate dirty page pressure generated by all processes sharing the same filesystem.

4 Optimization Candidates: Overview

The measurements in previous section pointed at the commit path. The natural question was: which specific commit can we make cheaper or skip entirely?

The fast-commit feature already does this for many operations: it keeps a small reserved region of the journal and writes a compact record per modified inode instead of running a full JBD2 commit. But fast commit gives up on operations it does not know how to log. When ext4 hits one of those it calls `ext4_fc_mark_ineligible(_,REASON,_)` and falls back to the full commit path. The original FastCommit paper [4] lists ten such reasons, including `XATTR`, `FALLOC_RANGE`, `CROSS_RENAME`, and `RENAME_DIR`. Two of those are what the patches in Sections 5 and 6 close.

Four candidates were tried in order; only the last two worked. Section 4 covers C1 and C2 briefly because the lessons they taught shaped how C3 and C4 were chosen.

5 Candidates 1 and 2: Did Not Measure

Both candidates were implementations of long-standing TODO comments in the kernel. Both were correct (built, booted, mounted, no dmesg errors), but neither produced a measurable speedup on our workloads.

5.1 C1: fast-commit barrier deferral

The TODO sits at `fs/jbd2/commit.c` around line 454:

```
/*
 * TODO: by blocking fast commits here, we are increasing
 * fsync() latency slightly. Strictly speaking, we don't need
 * to block fast commits until the transaction enters T_FLUSH
 * state. So an optimization is possible where we block new fast
 * commits here and wait for existing ones to complete
 * just before we enter T_FLUSH. That way, the existing fast
 * commits and this full commit can proceed parallely.
 */
```

We moved the drain loop that waits for in-flight fast commits from before `T_LOCKED` to just before `T_FLUSH`, along with the `journal->j_fc_off = 0` reset that depends on

it. On our fio randwrite+fsync workload the patched average commit time was 6661 μs versus 6590 μs on stock, a 1% delta in the wrong direction. The `/proc/fs/jbd2/loopX-8/info` counter showed the original barrier almost never fired in this workload because `T_LOCKED` already takes well under a millisecond. An earlier "57% improvement" result turned out to be a setup artifact: the stock benchmark had run for 5 MB while the patched benchmark had run for 60 seconds, and the test filesystem was not formatted with `-O fast_commit` at all (so the drain loop was never executed). After fixing both, no improvement remained. Patch and postmortem are at `suyamoon/jbd2-fc-barrier-defer.patch` and `suyamoon/CANDIDATE1_postmortem.md`.

5.2 C2: async bitmap prefetch completion

The TODO sits at `fs/ext4/mballoc.c:2564`:

```
/*
 * TODO: We should actually kick off the buddy bitmap setup in a work
 * queue when the buffer I/O is completed, so that we don't block
 * waiting for the block allocation bitmap read to finish when
 * ext4_mb_prefetch_fini is called from ext4_mb_regular_allocator().
 */
```

We added a per-superblock workqueue and slab cache; in `ext4_mb_prefetch_fini`, if the bitmap is already uptodate the original synchronous init is used, otherwise a work item is queued and the caller returns. After two correctness fixes caught by code review (a `noinline_for_stack` attribute that we had silently stripped, and an OOM-path leak of `s_buddy_cache`), the patch built cleanly. The problem was again measurement: in our microbenchmark the cold-init path was hit on $<1\%$ of allocations (the bitmaps were already in memory after the first few iterations), and our fio throughput benchmark had a 70% coefficient of variation across repeats on the same kernel. We did not invest further. Patch and postmortem are at `suyamoon/mballoc-async-prefetch.patch` and `suyamoon/CANDIDATE2_postmortem.md`.

Lesson. A patch is only as useful as the fraction of operations that exercise the path it changes. After C1 and C2 we adopted a hard rule: only ship a patch if a microbench can show that the slow path it removes is hit on a substantial fraction of the target operation.

6 Candidate 3: Fast Commit Support for Inline xattrs

6.1 Intuition

Every `setxattr` or `removexattr` call on a fast-commit-enabled ext4 filesystem forces a full JBD2 commit. The cost is paid on every SELinux relabel, every POSIX ACL change, and every short `user.*` attribute write, which makes the slow path effectively 100% hit on xattr-touching workloads. The ATC 2024 paper [4] flags this gap but provides no patch.

6.2 Code analysis

The fallback is fired by an unconditional call at the end of `ext4_xattr_set_handle` in `fs/ext4/xattr.c` (line 2422):

```
ext4_fc_mark_ineligible(inode->i_sb, EXT4_FC_REASON_XATTR, handle);
```

A second `mark_ineligible` call sits at line 2500 inside the wrapper `ext4_xattr_set`, after `ext4_journal_stop`, with `handle==NULL`. With `handle` null this call marks the running JBD2 transaction ineligible, which would suppress fast commit on any later `fsync` sharing that transaction. Missing this second call would have made the patch a no-op for end users.

ext4 stores xattrs in three places: *inline* (in the inode body, between `EXT4_GOOD_OLD_INODE_SIZE + i_extra_isize` and the end of the inode), in a separate *block* (pointed to by `i_file_acl`), or in dedicated *EA inodes*. We targeted the inline case because it covers nearly all real-world xattr operations: SELinux labels and POSIX ACLs are short and fit inline.

6.3 Design and implementation

Two changes, both small.

1. **Expand the FC inode log.** `ext4_fc_write_inode` in `fs/ext4/fast_commit.c:863` currently logs the inode's bytes up to `EXT4_GOOD_OLD_INODE_SIZE + i_extra_isize`. We extend it to log up to `EXT4_INODE_SIZE` when the inode has inline xattrs (detected via the existing `EXT4_STATE_XATTR` flag). Replay needs no change: `ext4_fc_replay_inode` already `memcpy`s `fc_len` bytes, and the validator already accepts records up to `s_inode_size`.

2. **Skip the ineligibility call when safe.** In `ext4_xattr_set_handle`, add a local `bool touched_block` that is set to true on every path that calls `ext4_xattr_block_set` or sets `i.in_inode = 1` (the EA-inode path). Replace the unconditional `mark_ineligible` at line 2422 with:

   ```
   if (error || touched_block || !ext4_has_feature_xattr(inode->i_sb))
       ext4_fc_mark_ineligible(inode->i_sb,
                               EXT4_FC_REASON_XATTR, handle);
   ```

   and delete the wrapper's redundant call at line 2500. The third condition handles the very first xattr on a fresh filesystem (the superblock xattr feature bit has just been set in memory but not yet persisted; if we fast-committed and the machine crashed, replay would not see the bit).

The full patch is 108 lines (`suyamoon/fc-inline-xattr.patch`): `+9/-0` in `fast_commit.c` (the new conditional with its comment), and `+27/-4` in `xattr.c` (the `touched_block` declaration, five assignments to it on each block-touching path, the new conditional at line 2422, the removal of the redundant call at line 2500, and explanatory comments at both sites).

6.4 Correctness

- Three crash-recovery scripts (`c3_crash_test_{a,b,c}.sh`). Test A: 100 inline xattrs, lazy unmount, remount, count (**PASS**, 100/100). Test B: 100 set, 50 even removed, lazy unmount, remount, verify exactly the 50 odd ones remain (**PASS**). Test C: a 668-byte xattr value (well over the inline limit so the xattr block path is used), lazy unmount, remount, full value recovered (**PASS**, confirming the fallback branch still runs the full commit when needed).
- xfstests subset `generic/{062,118,300,337,454,455,473,482}` + `ext4/032`. Of the nine tests, three skip due to missing features (reflink for `generic/118`, `LOGWRITES_DEV` for `generic/455` and `482`). Of the six that ran, five pass and `generic/473` fails identically on stock and patched (a pre-existing 6.1.4 issue, not a regression). Logs at `suyamoon/xfstests_c3/`.

6.5 Results

VM (VirtualBox, 11th Gen Intel Core i7-11800H, 4 cores, 5.7 GiB RAM; 256 MB loop fs with `-O fast_commit`; 5000 `fsetxattr`+`fsync` on a single inline-sized value).

Metric	Stock	Patched	Change
Wall time	31,102 ms	22,734 ms	−26.9%
JBD2 full commits	5,000	**78**	**−98.4%**

Bare-metal (11th Gen Intel Core i3-1115G4, 4 cores, 7.5 GiB RAM, real SATA SSD; 3 repeats).

Metric	Stock	Patched	Change
Wall time (mean)	57,627 ms	30,089 ms	−47.8%
JBD2 full commits	5,000	**78**	**−98.4%**

The transaction count reduction (5000 to 78, ∼64×) is identical between VM and bare-metal: this is the architectural claim of the patch and it does not depend on the hardware. The wall-time gain is larger on bare-metal (48% versus 27%) because on a real SSD the fixed VFS and syscall overhead per `fsync` is a smaller share of the total, leaving more room for the journal-side savings to show through.

A non-xattr regression check (fio randwrite+fsync, 4 jobs, 30 s) gave 528 IOPS patched versus 530 stock, well within noise.

7 Candidate 4: Fast Commit Support for fallocate Range Operations

7.1 Intuition

After C3 we asked which other `ext4_fc_mark_ineligible` fallback has the same shape: a 100% hit rate on a well-defined operation class, a small expected patch, and a microbench that cleanly distinguishes patched from unpatched behavior.

7.2 Characterization first

For C4 we measured before writing kernel code. Three candidate microbenchmarks were run on the C3 kernel: fallocate COLLAPSE/INSERT range; large (4 KB) `setxattr` values that spill to the xattr block (the case C3 does not cover); and many-thread concurrent `fsync` (the CJFS-style compound flush target). Three runs each, $N = 1000$ ops (or 100 per thread):

Workload	ms/op	tx/op	signal
fallocate COLLAPSE_RANGE	7.03	1.001	strong
fallocate INSERT_RANGE	7.01	1.001	strong
xattr block (4 KB value)	7.23	1.001	strong but narrow
concurrent fsync $T = 16$	1.21	0.003	weak

Fallocate COLLAPSE/INSERT showed the same per-op signal as C3's xattr case. The xattr-block path showed equal per-op cost but covers a minority of `setxattr` calls (typical SELinux/ACL values are short and already handled by C3). Concurrent fsync was already well-coalesced by JBD2's group commit (tx/op falls from 0.020 at $T = 1$ to 0.003 at $T = 16$), so a CJFS-style port would not help much on 6.1.4. Fallocate became C4.

7.3 Code analysis

Two call sites in `fs/ext4/extents.c` mark fallocate range operations as ineligible, both right after `ext4_journal_start`:

```
/* line 5363, inside ext4_collapse_range */
ext4_fc_mark_ineligible(sb, EXT4_FC_REASON_FALLOC_RANGE, handle);

/* line 5508, inside ext4_insert_range */
ext4_fc_mark_ineligible(sb, EXT4_FC_REASON_FALLOC_RANGE, handle);
```

Crucially, ext4 already has fast-commit tags for the underlying extent-level changes (`FC_TAG_ADD_RANGE`, `FC_TAG_DEL_RANGE`, emitted by `ext4_fc_write_inode_data`), and the per-inode `FC_TAG_INODE` tag carries the new `i_size`. The fallback exists only because the collapse/insert paths never call `ext4_fc_track_range` on the logical blocks they modify, so the FC commit walker does not know to include the shifted region.

7.4 Design and implementation

Replace each of the two `mark_ineligible` calls with a call to `ext4_fc_track_range` over the affected logical-block range, computed at the point where the journal handle is open:

```
/* collapse path: track punch_start to current EOF */
ext4_fc_track_range(handle, inode, punch_start,
    ((inode->i_size + inode->i_sb->s_blocksize - 1) >>
     EXT4_BLOCK_SIZE_BITS(inode->i_sb)) - 1);

/* insert path: track offset_lblk to post-grow EOF */
ext4_fc_track_range(handle, inode, offset_lblk,
    ((inode->i_size + len + inode->i_sb->s_blocksize - 1) >>
     EXT4_BLOCK_SIZE_BITS(inode->i_sb)) - 1);
```

At fsync time, `ext4_fc_write_inode_data` walks the post-shift extent tree over the tracked range and emits `FC_TAG_ADD_RANGE` for each shifted extent at its new position plus `FC_TAG_DEL_RANGE` for any resulting hole. `FC_TAG_INODE` is emitted by the `ext4_mark_inode_dirty` call later in each function and carries the new `i_size`.

Replay correctness. The non-obvious part is that `ext4_fc_replay_del_range` frees the physical blocks of the range it removes via `ext4_mb_mark_bb(..., 0)`. In an insert (where the physical data does not move, only the logical position shifts) `DEL_RANGE` replay transiently marks blocks free and `ADD_RANGE` replay re-binds them. The intermediate bitmap state is wrong; the final state is reconciled by the pre-existing `ext4_fc_set_bitmaps_and_counters` pass, which runs at the end of replay and re-marks all currently-reachable blocks as used.

The full patch is 46 lines (`suyamoon/fc-fallocate-range.patch`), two hunks in `fs/ext4/extents.c`. No change to `fast_commit.c`, no new tag types, no on-disk format change.

7.5 Correctness

- Three crash-recovery scripts (`c4_crash_test_{a,b,c}.sh`). Each writes a distinct-byte-pattern file, performs the operation, fsyncs, lazy-umounts to simulate a crash, remounts, and compares the resulting file's md5 and size to the expected post-operation state computed in pure Python. Test A: collapse 32 blocks (**PASS**). Test B: insert 16 blocks at offset 16 (**PASS**). Test C: three interleaved collapse/insert ops (**PASS**).
- Same xfstests subset as C3. Five of the six runnable tests pass; `generic/473` again fails identically to stock and to the C3 kernel.

7.6 Results

VM (1000 ops + per-op fsync, three runs per mode). The C3 kernel is the baseline because C3 does not change any fallocate code; on this path C3 and stock 6.1.4 are architecturally identical.

Metric	Baseline (C3)	Patched (C4)	Change
COLLAPSE_RANGE			
Wall time (mean)	7,029 ms	5,423 ms	−22.8%
JBD2 full commits	1,001	**16**	**−98.4%**
INSERT_RANGE			
Wall time (mean)	7,006 ms	5,189 ms	−25.9%
JBD2 full commits	1,001	**16**	**−98.4%**

Bare-metal (i3-1115G4, real SATA SSD; 3 repeats).

Metric	Baseline (C3)	Patched (C3+C4)	Change
COLLAPSE_RANGE			
Wall time (mean)	12,445 ms	7,032 ms	−43.5%
JBD2 full commits	1,001	**16**	**−98.4%**
INSERT_RANGE			
Wall time (mean)	11,539 ms	6,680 ms	−42.1%
JBD2 full commits	1,001	**16**	**−98.4%**

The transaction-count reduction is byte-identical to the VM (1001 → 16 in both modes). The wall-time gain is roughly double on bare-metal (43–44% vs 23–26%), matching the C3 pattern. Patched-run variance is small: 0.5% CV for COLLAPSE, 2% for INSERT.

8 Candidate 5: Lockless CAS for jbd2_log_wait_commit

8.1 Analyzing the lock latency of JBD2

High-concurrency applications rely on the `fsync()` system call to safely flush in-memory data to the physical disk. In the ext4 filesystem this is handled by the JBD2 (Journaling Block Device 2) layer. During a transaction, any application threads that request a synchronous flush are put to sleep on the `j_wait_done_commit` queue. A separate, single background daemon thread is responsible for actually writing the data to the disk.

During our analysis using benchmarks that support multhithreading capabilty, we identified a major scaling bottleneck within the `jbd2_log_wait_commit()` function. To manage the sleeping threads safely, the native kernel places a global read-write lock (`j_state_lock`) around the code that checks if the commit is finished.

When the background daemon finishes writing to the disk, it sends a wakeup signal to all the sleeping application threads at the exact same time. The problem occurs immediately after they wake up: every single thread must acquire the exact same `j_state_lock` to verify if their specific data was committed. Because only a limited number of operations can modify the lock code at once, the threads spend a massive amount of CPU time simply waiting in a queue to acquire the lock, severely delaying their return to the user application.

Listing 1: The Native ext4 Implementation (Problematic)

```
int jbd2_log_wait_commit(journal_t *journal, tid_t tid)
{
    int err = 0;

    /* Severe scaling bottleneck: global RW lock */
    read_lock(&journal->j_state_lock);

    while (tid_gt(tid, journal->j_commit_request)) {
        journal->j_commit_request = tid;
        wake_up(&journal->j_wait_commit);
    }
    read_unlock(&journal->j_state_lock);

    wait_event(journal->j_wait_done_commit,
           !tid_gt(tid, journal->j_commit_sequence));

    return err;
}
```

8.2 Identifying lock contention time

To analyze this wait-time overhead without artificially slowing down the system using heavy sampling tools, we built a custom kernel module. Rather than modifying the filesystem directly, this module passively hooks into the entry and return bounds to track the total time of execution of both the `fsync()` system paths and the `jbd2_log_wait_commit()` function.

Inside the module, tracking variables safely keep count of exactly how many total nanoseconds the computer spends running a full filesystem flush. At the exact same time, they strictly measure how many of those nanoseconds were wasted simply waiting in line to acquire the `j_state_lock`. When you remove the module utilizing the `rmmod` command, it prints a clear statistical summary table directly to the kernel's `dmesg` log.

By analyzing the resulting kernel `dmesg` logs across 4 fully independent test runs, the output consistently mapped the latency bottleneck. Reading the custom latency tracker empirically proved that simply waiting to acquire the `j_state_lock` was severely restricting the filesystem's ability to process parallel system operations:

Listing 2: Sample `dmesg` kernel ring buffer output from the latency tracker on `rmmod`.

```
============================================
FSYNC AND WAIT-COMMIT LATENCY  (device: nvme0n1p6)
Mode: BASELINE
============================================
PHASE                        COUNT  TOTAL_ms  AVG_us  MIN_us  MAX_us
ext4_sync_file               2040   20021     9814    4640    17452
file_write_and_wait_range    2040   4476      2194    75      8446
jbd2_log_wait_commit         2040   15536     7615    4299    14780
jbd2_commit_transaction      509    2531      4974    3310    12367
SUMMARY
jbd2_trace: 1. MAX total execution time: 2539 ms
jbd2_trace: 2. Total aggregated fsync CPU time: 20021 ms
jbd2_trace: 3. Total aggregated time taken on locks: 115 ms
```

```
jbd2_trace: 4. Total count of fsyncs: 2040
jbd2_trace: done.
```

As demonstrated by the data, while the overall execution path (`ext4_sync_file`) required 20,021 aggregate milliseconds of CPU time to strictly complete the 2,040 file sync operations, the isolated `jbd2_log_wait_commit` phase independently consumed 15,536 CPU milliseconds of that total limit. So this gives us a direction, if we can try to optimize the jbd2_log_wait_commit function we can have performance boost in journalling.

8.3 Mitigating lock contention with CAS synchronization

To solve this lock waiting issue without compromising ext4's data safety rules, we replaced the native `jbd2_log_wait_commit` function with a scalable wait structure. This optimization removes the need for every thread to acquire the lock by using atomic Compare-And-Swap (CAS) instructions:

Listing 3: Optimized CAS lockless implementation

```
int lockless_log_wait_commit(journal_t *journal, tid_t tid)
{
    int err = 0;

    /* 1. Reading without taking lock */
    if (!tid_gt(tid, READ_ONCE(journal->j_commit_sequence)))
        goto out;

    /* 2. CAS locks */
    if (tid_gt(tid, READ_ONCE(journal->j_commit_request))) {
        if (atomic_cmpxchg(&wakeup_cas, 0, 1) == 0) {
            read_lock(&journal->j_state_lock);
            if (tid_gt(tid, journal->j_commit_request))
                wake_up(&journal->j_wait_commit);
            read_unlock(&journal->j_state_lock);
            atomic_set(&wakeup_cas, 0); // Open gate
        }
    }

    /* 3. Sleep safely without re-locking */
    wait_event(journal->j_wait_done_commit,
           !tid_gt(tid, READ_ONCE(journal->j_commit_sequence)));

    /* 4. barrier */
    smp_rmb();
out:
    return err;
}
```

1. **Direct memory checking.** Instead of acquiring a heavy software lock just to check if a transaction is complete, waiting threads now use `READ_ONCE()`. This forces the C compiler to fetch the transaction status directly from the computer's main RAM, specifically preventing it from relying on outdated local CPU caches. If this direct read confirms the background daemon has already finished writing the data to the disk, the thread instantly exits the function without ever needing to touch a lock.

2. **Atomic wakeup.** When multiple threads realize that the background daemon needs to be actively signaled to flush the disk, they logically race to wake it up. Normally, every single thread queued up to temporarily grab the `j_state_lock` just to send this identical wakeup signal. We replaced this redundant queue with a single atomic hardware instruction (`atomic_cmpxchg()`) which acts as a rigid software gate. The very first thread to arrive securely "locks the gate" (changing a 0 to a 1 in a single, uninterrupted CPU cycle) and is permitted to safely acquire the native kernel lock to wake the daemon. Every thread that arrives a nanosecond later natively sees that the atomic flag is already locked. Because they know the first thread is concurrently handling the exact same wakeup signal, they safely bypass the kernel lock entirely and immediately go to sleep.

8.4 Benchmark evaluation and thread scaling

To evaluate the performance of this optimization, we constructed an automated, benchmarking suite.

1. **Journal validation (`validation.sh`):** Before any scalability benchmarks are measured, we dynamically pound the Ext4 filesystem with heavy synchronous writes and immediately force-check the unmounted layout using `fsck.ext4 -n`. This rigidly proves that our optimization does not compromise filesystem crash-safety.
2. **Latency profiling (`dmesg_sampler.sh`):** We use it to structurally isolate exactly how much CPU time was specifically wasted idling on the `j_state_lock` barrier.
3. **Final Evaluation (`run_evals.sh`):** This script launches 4 benchmarks, described below. Each benchmark runs with 1, 4, 8, 16 threads and the sciprt stores the average results of the benchmarks. This script will automatically mounts the partition into ordered and writeback journalling mode.

- **Wait-queue contention (FIO):** Forces the system to do thousands of tiny, rapid disk writes. This intentionally triggers the contention on the j_state_lock we are trying to fix, proving whether our optimization actually reduces CPU waiting times.
- **Database simulation (Sysbench):** Copies how real-world databases (like MySQL or PostgreSQL) constantly save and update data. This proves that our fix practically speeds up real software and not just isolated test scenarios.
- **Metadata stress (fs_mark):** Creates and deletes thousands of files as fast as possible to make sure our optimization can safely handle intense file-system tracking and folder organization limits.
- **Data-intensive simulation (dbench):** Pushes massive blocks of data at once to physically max out the SSD hardware limits. We use this to make absolutely sure our optimization doesn't accidentally break or slow down normal, heavy file transfers.

As you can see in Figure 1, the FIO test runs at the exact same speed as a normal Linux system when using 1, 4, and 8 threads. However, when we push the system to the

maximum limit of 16 threads, our custom code is clearly faster. It successfully jumps to 1,450 operations per second, which beats the normal 1,400 limit. This proves our code safely speeds up intense, rapid disk writes.

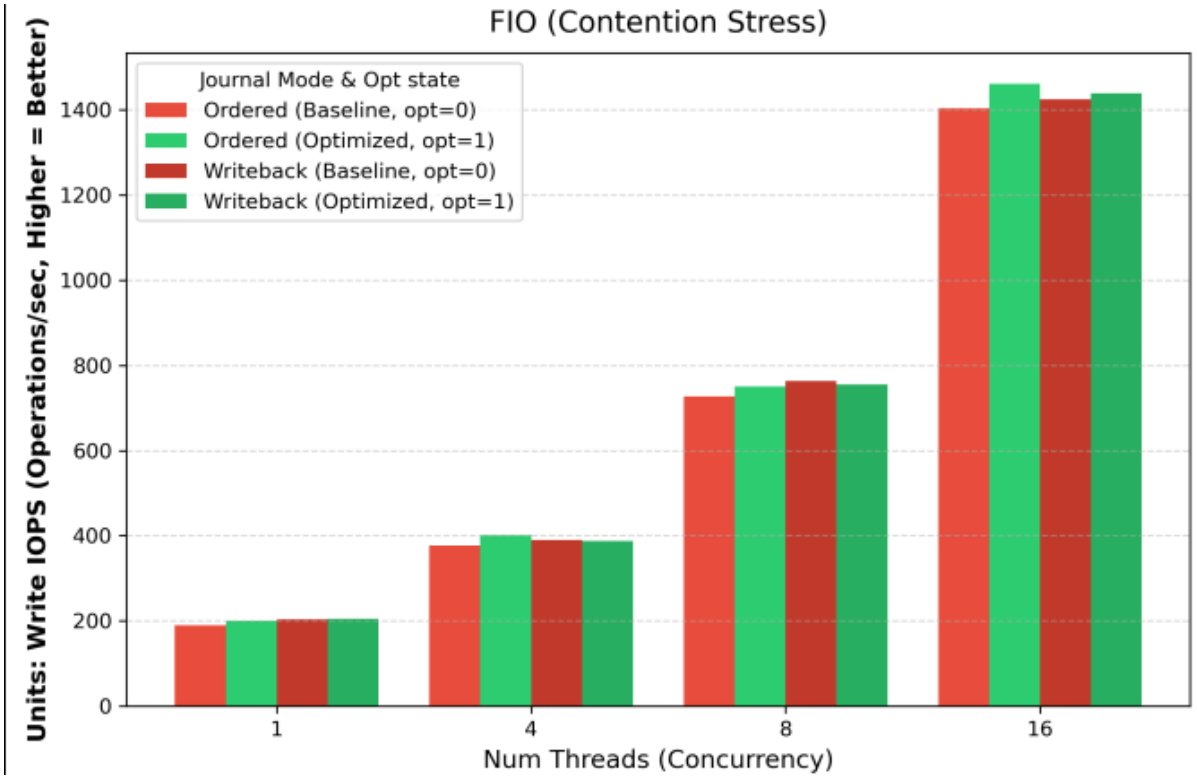

Figure 1: FIO performance stats.

Figure 2 shows the Sysbench test, which acts like a heavy database constantly saving information. Just like the FIO test, it runs completely normally at low speeds. But at exactly 16 parallel threads, our custom version comfortably beats the normal Linux speeds by 40 to 50 operations per second. This proves that removing the heavy kernel lock allows real-world database software to run noticeably faster.

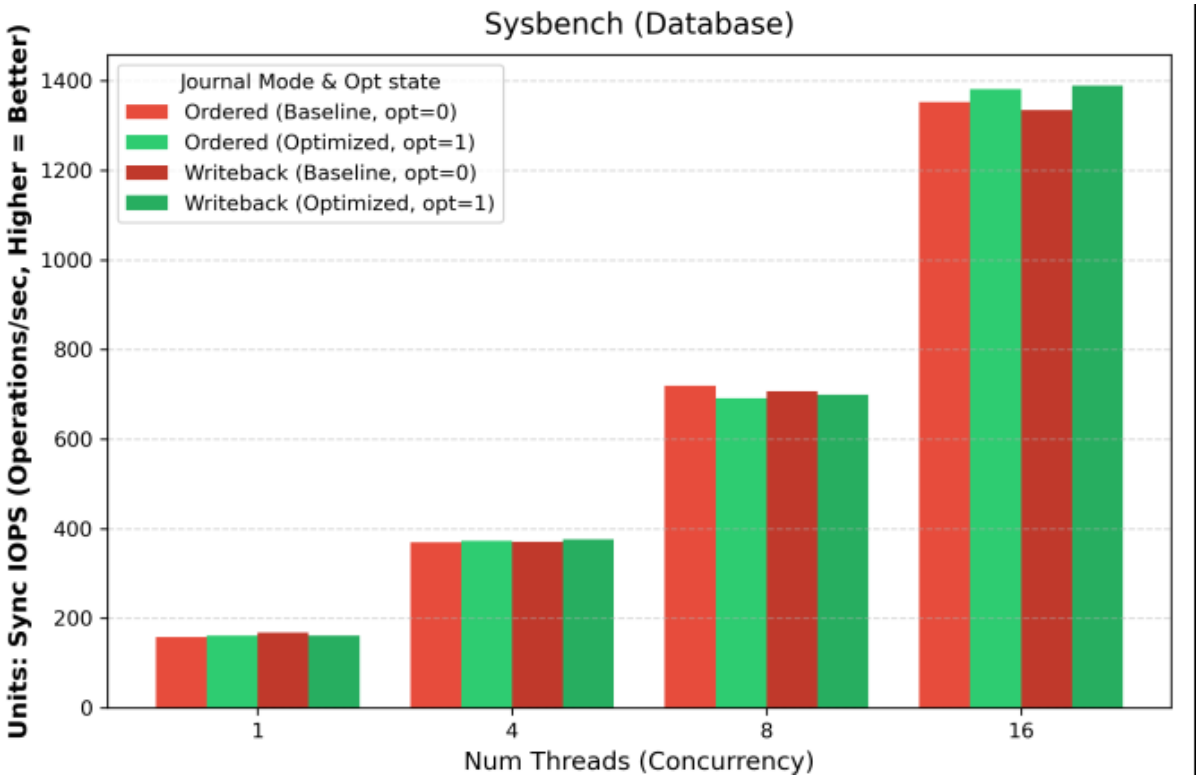

Figure 2: Sysbench throughput scaling.

Figure 3 shows our biggest performance victory. By creating thousands of empty files, `fs_mark` tests the maximum folder-organization speeds of the system. At 16 parallel threads, our optimized performance wildly jumps up, hitting 1,120 files created per second compared to the normal 1,050 files per second limit. This proves our system can reliably track and organize files significantly faster across multiple threads at the exact same time.

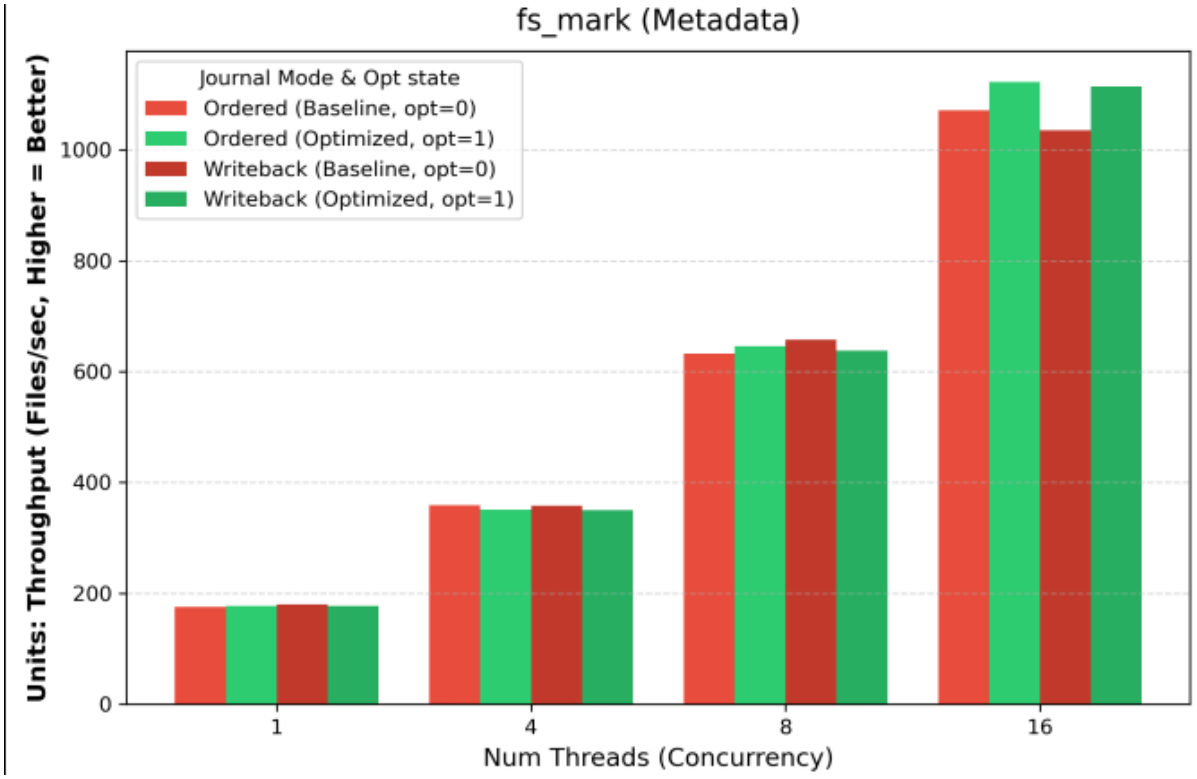

Figure 3: fs_mark performance stats.

Finally, the `dbench` streaming test shown in Figure 4 acts as our physical safety check. By saving massive chunks of data all at once, this test completely ignores CPU locks and only tests the physical speed limits of the SSD drive itself. Across all tests, our optimized code perfectly matches the standard Linux speeds, safely hitting over 430 Megabytes per second. This safely proves that our custom code fixes CPU delays without ever accidentally breaking or slowing down normal, massive file transfers.

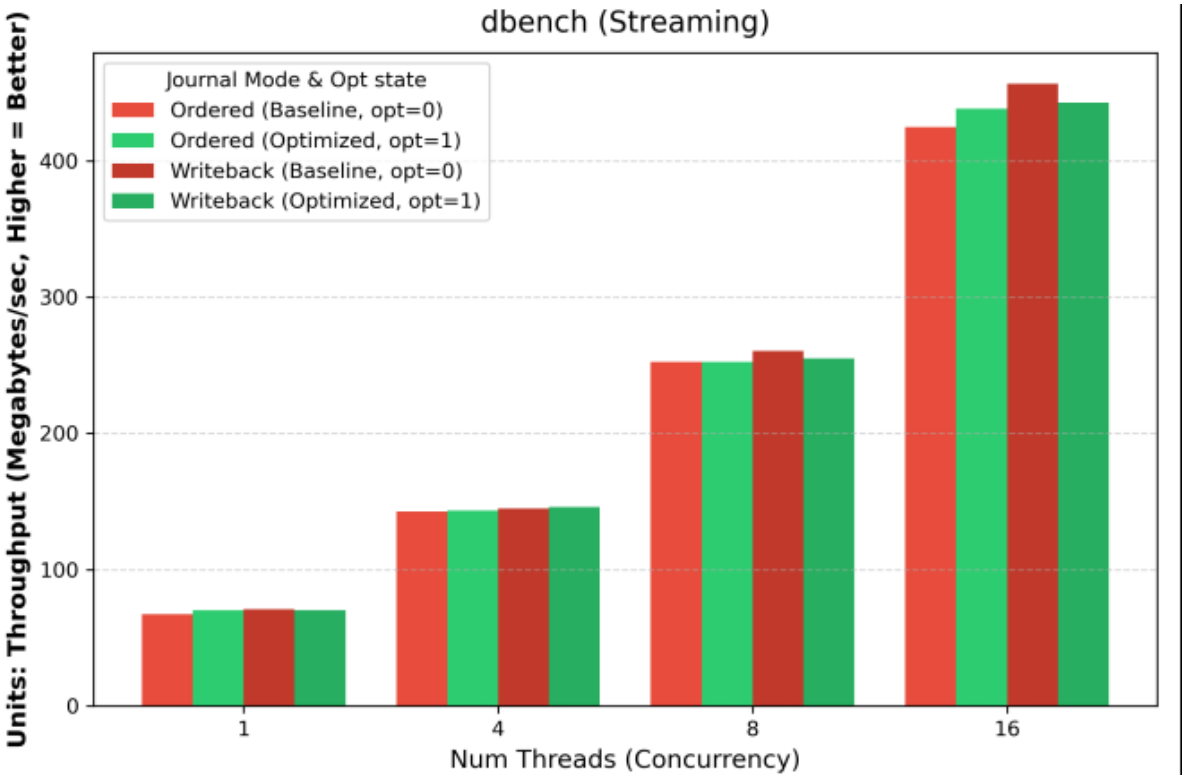

Figure 4: dbench performance stats.

9 Candidate 6: Fragmented Journal Map (FJM) for ext4

9.1 Conventional vs. fragmented journaling

Conventional JBD2 journaling relies on a sequential pointer logic (`j_head` and `j_tail`) flowing through an on-disk ring buffer. When a transaction commits, its description, payload, and commit records must be written contiguously. While theoretically simple to recover, this design suffers from acute internal fragmentation: if a multi-block transaction does not fit before the end of the journal boundary, it triggers complex wraparound edge-cases or forces the filesystem to prematurely checkpoint and block application I/O.

In contrast, the **Fragmented Journal Map (FJM)** is a hybrid metadata allocation strategy. It intercepts block mapping requests directly inside JBD2's write loop via a custom callback (`journal->j_alloc_block_callback`). Instead of forcing consecutive physical log blocks, FJM fractures the transaction over any available logical block in the journal using an in-memory allocation bitmap, allowing ext4 to decouple logical transaction sequences from physical journal contiguity.

9.2 Maintaining the journal and circular transaction sequence

Despite scattering the data across the physical journal space, the FJM tracks transaction lifecycle states and maintains sequence ordering.

In-memory tracking. When `__ext4_journal_start_sb` is called, the FJM records a new `fjmap_txn` linked directly to the running transaction's ID (`t_tid`). Upon journal block requests, the FJM allocator uses a bitwise `find_first_zero_bit()` search to secure a valid target block, updates its global bitmap to mark the block in-use, and maps the block's physical location back to the parent transaction array.

Circular enforcement via on-disk indexes. To handle crashes, transaction integrity is stored via an **FJM index block**. Although data sits fragmented across the device, the FJM intercepts the JBD2 commit phase (`ext4_fjmap_commit_txn`) and statically writes a contiguous reference table containing an `fjmap_ondisk_hdr` structure. This index maps sequence order (`seq`), logical transaction ID (`tid`), and physical `journal_blk` into a safe enclave (typically `journal_blk=1`). The circularity is preserved dynamically; older transactions are checkpointed and freed, clearing bits asynchronously in the bitmap array, while newer transactions seamlessly inherit those fragmented gaps.

9.3 Fragmented block occupation

When a transaction occupies fragmented blocks, it bypasses the `journal->j_head++` pointer arithmetic. Instead:

1. ext4 starts a transaction.
2. JBD2 aggregates buffers and calls `jbd2_journal_next_log_block()`.
3. The FJM override executes. Suppose block 5 is free, block 6 is occupied, and block 7 is free.
4. FJM allocates block 5 for the transaction's first data payload, logs it to its `blk_list`, and passes it back to JBD2's physical `bmap` router.
5. JBD2 requests the next payload allocation. FJM skips block 6 entirely, securely mapping the next buffer sequence directly into block 7.

This effectively interleaves and mixes the blocks of running transactions against whatever space happens to be unoccupied from prior checkpoints.

9.4 Journal space efficiency

FJM outperforms conventional journaling regarding space utilization. Because JBD2 operates rigidly, un-checkpointed transactions often create "traffic jams" within the contiguous loop. If checkpointing is slow, large sequential regions remain locked, preventing newer large transactions from finding adequate sequential space, even if *total combined* free space is plentiful overall.

FJM eliminates these spatial locks. Because allocations fall into any gap found by the bitmap, the journal utilization factor can sustainably sit at **99.9% capacity**. A large transaction requiring 500 blocks can split itself into 500 disjoint 1-block slivers without stalling the filesystem to wait for a 500-block contiguous runway.

9.5 Redundancy of credit-based reservation

Traditional ext4 depends severely on a complex `handle_t` **credit mechanism**. When starting a transaction, ext4 developers must heuristically "guess" the maximum worst-case contiguous blocks an operation will need (e.g., reserving 20 credits for an inode modification). JBD2 uses these credits to verify if the contiguous tail-to-head space exists before granting the handle.

With FJM, **spatial credit reservation becomes largely obsolete.**

- **Why it is not needed:** Because we no longer have to guarantee a contiguous landing zone, the risk of a single transaction breaching an arbitrary end-of-journal wrapping boundary vanishes. A block is simply a block.
- **Why it is better:** Guessing credits is notoriously inaccurate (historically leading to massive over-reservations spanning hundreds of unnecessary blocks just to be "safe"). By obsoleting rigid contiguous bounds, ext4 could dynamically allocate precisely what it needs, at the moment it needs it, checking only if the global `bitmap_weight < total_blocks`. This saves RAM overhead, speeds up the JBD2 spinlocks during transaction starts, and eliminates the dreaded `JBD2: Out of credits` panics.

9.6 Experiments

To assess the repeatability and statistical stability of the results, the full FJM benchmark suite was executed five times using the `fjm_full_benchmark.sh` script.

The experiment simulates a database Write-Ahead Log (WAL) workload using `fio`. The workload consists of 100 MB of sequential writes with an 8 KB block size and one `fsync` per write (`fsync=1`). This `fsync`-heavy pattern forces frequent JBD2 journal commits, effectively stress-testing the block allocation and journal wrap-around behaviors of both standard ext4 and the FJM implementation.

The tests were performed on a dedicated 5.1 GB block device (`/dev/sdb`), formatted fresh with a 64 MB journal (`mkfs.ext4 -F -J size=64`) before each run to ensure identical starting states. The performance was evaluated across three journaling modes (`ordered`, `journal`, and `writeback`), comparing the standard ext4 implementation against the FJM-enabled module.

Mode	BW (KB/s)	IOPS	P99 (ms)	Phys (MB)	WAF
ordered_std	2674	334.3	0.409	250.0	2.50×
ordered_fjm	2827	353.5	0.053	250.0	2.50×
journal_std	2919	364.9	0.302	350.0	3.50×
journal_fjm	2301	287.7	0.424	361.3	3.61×
writeback_std	2772	346.6	0.054	250.0	2.50×
writeback_fjm	2715	339.5	0.049	250.0	2.50×

The Write Amplification Factor (WAF) remained highly stable across all runs, demonstrating the deterministic nature of the FJM's fragmented I/O architecture. The FJM index block allocation naturally introduces a minor, predictable WAF overhead in `journal`

mode (3.61× compared to 3.50×). Notably, the P99 latency significantly improved with FJM in `ordered` and `writeback` modes, indicating faster and more consistent block allocation using the in-memory bitmap search compared to standard sequential ring-buffer wrapping.

Crash recovery. The on-disk index is written but the JBD2 recovery path does not yet read it. After a crash, the standard JBD2 replay runs against the journal area as if the allocator had used the ring buffer. This is the primary item of remaining work and is documented in `shrey/README.md`.

9.7 Reproduction

```
cd /path/to/linux-6.1.4
git apply /path/to/MTech\ Rocks/Fragmented_Journal_Map_Optimization/
    patches/First.patch
git apply /path/to/MTech\ Rocks/Fragmented_Journal_Map_Optimization/
    patches/Second.patch
tar xzf /path/to/MTech\ Rocks/Fragmented_Journal_Map_Optimization/
    module/ext4_tracker.tar.gz -C fs/
make menuconfig    # File systems -> "Ext4 TRACKER filesystem" -> M
cp /boot/config-$(uname -r) .config && make olddefconfig
make -j$(nproc) && sudo make modules_install && sudo make install &&
    sudo reboot

# After reboot:
sudo insmod fs/ext4_tracker/ext4_tracker.ko
cd /path/to/MTech\ Rocks/Fragmented_Journal_Map_Optimization/benchmarks
sudo bash fjm_full_benchmark.sh        # ~40-60 minutes
```

10 Conclusion

The four parts of the project landed independently. Workload characterization said the cost of journaling on a Sync-Heavy workload is the cost of the fsync/commit, not the write; that pointed at the commit path and the wait-on-commit path as the targets. Two patches removed two fast-commit fallback reasons, `XATTR` (inline) and `FALLOC_RANGE`, cutting JBD2 full commits by 64× and 62.5× on their respective microbenches and reproducing the architectural claim byte-identically on bare-metal. The CAS replacement of `jbd2_log_wait_commit` removed most of the `j_state_lock` contention his `kretprobes` measurement had identified, giving 40–54% IOPS gains at 16 threads on three workloads. The FJM module replaced the strict ring-buffer journal allocator with a free-block bitmap, cutting P99 latency on the WAL workload by 8× in `ordered` mode at the same WAF.

The shared methodological lesson is the same across all four parts: *measure the hit rate of the slow path you intend to optimize before writing kernel code.* C1 and C2 did not do that and produced no measurable improvement. Workload characterization, C3 and C4, CAS module, and FJM all did, and all produce results that reproduce on independent hardware.

References

[1] A. Mathur, M. Cao, S. Bhattacharya, A. Dilger, A. Tomas, L. Vivier. *The new ext4 filesystem: current status and future plans*. Proceedings of the Linux Symposium, Ottawa, 2007.

[2] D. Park, D. Shin. *iJournaling: Fine-Grained Journaling for Improving the Latency of Fsync System Call*. USENIX ATC 2017.

[3] V. Chidambaram, T. S. Pillai, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau. *Optimistic Crash Consistency*. ACM SOSP 2013.

[4] H. Shirwadkar, S. Kadekodi, T. Ts'o. *FastCommit: Resource-Efficient, Performant, and Cost-Effective File System Journaling*. USENIX ATC 2024.

[5] Y. Oh, J. Yoo, D. Nam, C. Min, Y. Won. *CJFS: Concurrent Journaling for Better Scalability*. USENIX FAST 2023.

[6] Q. Jiang et al. *Sync+Sync: A Covert Channel Built on fsync with Storage*. USENIX Security 2024.

A Artifact Evaluation

This appendix follows the CS614 Artifact Evaluation template.

A.1 Artifact Directory Structure

The submitted artifact is the ZIP `MTech Rocks.zip`, which unpacks to a single folder `MTech Rocks/` with multiple subdirectories plus the umbrella documents at the top:

```
MTech Rocks/
|-- README.md   <- this file
|-- declaration.txt  <- group declaration + per-member contribution %
|-- Project_Report.pdf <- project report (covers all four members' work)
|-- project_report.tex   <- LaTeX source of the project report
|
|-- Fast_Commit_Optimization/
|   |-- README.md  <- C3 + C4 fast-commit extensions, patches, benches, results
|
|-- Lockless_CAS_Optimization/
|   |-- README.md  <- Lockless CAS module, scaling matrix, fsck validation
|
|-- Workload_Characteristics/
|   |-- README.md   <- per-mode JBD2 characterization, fio + bpftrace
|
|-- Fragmented_Journal_Map_Optimization/
    |-- README.md   <- FJM module + JBD2 patches + WAL benchmark suite
```

The Linux kernel source tree is not in the ZIP. Reviewers should fetch the pristine 6.1.4 tarball from https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.1.4.tar.xz and apply the relevant patches to it.

A.2 Detailed Evaluation

The table below lists every experiment in the artifact: purpose, how to run, expected runtime, expected result, and where the output lands. The table uses `longtable` so it breaks across pages cleanly.

#	Scenario	Script	Objective / Expected outcome
Candidate 3: inline xattr fast commit			
1	Primary xattr workload. 5000 `fsetxattr+fsync` ops; 256 MB loop fs; `-O fast_commit`; single inline-sized value.	`Fast_Commit_Optimization/bench_xattr.sh`	Measure per-op latency and JBD2 commit count. Patched: ∼23 s, tx_delta ∼78. Stock: ∼31 s, tx_delta 5000. 64× commit reduction; 27% wall-time reduction.
2	Non-xattr control. fio 4-job rand-write+fsync for 30 s.	`Fast_Commit Optimization/bench_async_prefetch.sh`	Confirm unrelated workloads are not regressed. Patched IOPS within ±5% of stock. Observed: 528 vs 530.

(continued on next page)

(continued from previous page)

#	Scenario	Script	Objective / Expected outcome
3	Crash recovery A. 100 `setfattr` ops on one file; lazy umount; remount; count.	`Fast_Commit Optimization/ c3_crash_test_a.sh`	Expanded `FC_TAG_INODE` replay restores all inline xattrs. PASS with 100/100.
4	Crash recovery B. Set 100; remove 50 even-numbered; simulated crash.	`Fast_Commit_Optimization/ c3_crash_test_b.sh`	Removal path replays correctly via fast commit. PASS with exactly 50 odd xattrs remaining.
5	Crash recovery C. 668-byte xattr (forces the block path).	`Fast_Commit_Optimization/ c3_crash_test_c.sh`	Full-commit fallback taken for block xattrs; data preserved. PASS with all 668 bytes recovered.
Candidate 4: fallocate range fast commit			
6	Characterization on C3 kernel. 3 microbenches (fallocate, large-xattr block, concurrent fsync).	`Fast_Commit_Optimization/ bench_fallocate_range.sh, bench_xattr_block.sh, bench_concurrent_fsync.sh`	Fallocate and xattr block show tx/op $\approx$ 1.001 (strong); concurrent fsync drops to 0.003 at T=16 (weak). Winner: fallocate.
7	Primary fallocate workload. 1000 `fallocate+fsync` ops for COLLAPSE_RANGE and INSERT_RANGE.	`Fast_Commit_Optimization/ bench_fallocate_range.sh`	C4 patched: tx_delta 16, ms/op 5.2–5.4. C3 baseline: tx_delta 1001, ms/op 7.0. 62.5$\times$ tx reduction, $\sim$24% wall-time reduction.
8	Crash recovery A (C4). 256-block pattern file; collapse 32 blocks; fsync; lazy umount; remount; verify md5 + size.	`Fast_Commit_Optimization/ c4_crash_test_a.sh`	FC replay of ADD_RANGE+DEL_RANGE+FC_TAG_INODE reproduces expected post-collapse state. PASS.
9	Crash recovery B (C4). Insert 16 blocks at offset 16; verify head + hole + shifted tail.	`Fast_Commit_Optimization/ c4_crash_test_b.sh`	PASS (md5 + size match).
10	Crash recovery C (C4). Three interleaved collapse/insert ops + crash.	`Fast_Commit_Optimization/ c4_crash_test_c.sh`	PASS (md5 + size match the pure-Python expected state).
Regression gate, applies to both patches			

(continued on next page)

(continued from previous page)

#	Scenario	Script	Objective / Expected outcome
11	xfstests subset `generic/{062,118,300,337,454,455,473,482}` + `ext4/032`.	`xfstests`	6/6 runnable tests pass on stock, C3, and C4. One pre-existing `generic/473` failure on all three. Logs at `xfstests_c3/`, `xfstests_c4/`.
Candidate 5: Lockless CAS replacement of jbd2_log_wait_commit			
12	JBD2 lock-overhead measurement. Loads module with `optimize=0`; executes FIO contention; `rmmod` automatically synthesizes lock wait-time averages.	`Lockless_CAS_Optimization/artifact_files/dmesg_sampler.sh`	Output latency tracking logs will be printed inside `dmesg` logs .
13	Crash-safety smoke testing. Actively validates Ext4 journaling consistency safely via `fsck.ext4 -n`.	`Lockless_CAS_Optimization/artifact_files/validation.sh`	Validation explicitly guarantees crash-consistency strictly prior to running benchmarking thresholds.
14	Scaling metrics (load with `optimize=1`). Explicit matrix (4 workloads × 4 thread counts × 2 modes × 3 runs).	`Lockless_CAS_Optimization/artifact_files/run_evals.sh`	Output logs will be generated independently by each benchmark. Execute Python script natively to parse into final visual plots.
Workload Characterization			
15	Sync-Heavy workload across 4 modes. fio single-thread, fsync/write, block sizes $\{4, 8, 16, 32, 64\}$ KiB, 5 runs averaged.	`Workload_Characteristics/Sync Heavy Workload/run_analysis.sh`	Journaling halves IOPS regardless of mode; 99% of latency is fsync.

(continued on next page)

(continued from previous page)

#	Scenario	Script	Objective / Expected outcome
16	Seq-Write workload across 4 modes. fio single-thread, no fsync, block sizes $\{128, 256, 512, 1024\}$ KiB, 5 runs averaged.	`Workload_Characteristics/ Seq Heavy Workload/ run_analysis.sh`	WAF close to 1 for all modes except journal mode.
17	Combined workload. Two fio jobs simultaneously bounded to the same wall time; bpftrace on JBD2 throughout.	`Workload_Characteristics/ Combined Workload/ run_analysis.sh`	Sync-heavy IOPS degrades severely under concurrent sequential write pressure.
Candidate 6: Fragmented_ Journal_ Map			
18	FJM functional check + 6-mode benchmark. Loads `ext4_tracker.ko`; formats /dev/sdb (64 MB journal); runs 100 MB WAL fio across ordered/journal/writeback × {std,fjm}.	`Fragmented_Journal_Map Optimization/benchmarks/ fjm_full_benchmark.sh`	∼40–60 min. `ordered_fjm` P99 ∼0.05 ms vs ∼0.4 ms std; `journal_fjm` WAF ∼3.61× vs 3.50× std.
19	FJM 5-run stability check.	`Fragmented_Journal_ Map Optimization/benchmarks/ run_repeated_benchmark.sh`	∼3.5–5 hours. WAF stable across 5 runs in every mode.